

Token Diet: Input and Output Compression for LLM API Cost Reduction

Maksimilian Gromovoi
HSE University
Master's in Data Science
mrgromovoy@edu.hse.ru

1 Introduction

LLM APIs charge per token. For tasks like summarization, cost accumulates quickly when you process thousands of documents. We study two compression strategies that reduce token count without changing the underlying model:

- Input compression: mechanically stripping filler words, determiners, and light verbs from the source text before sending it to the model.
- Output compression: prompting the model to respond in a concise style (“caveman-speak”) that drops articles, hedging, and filler.

Both approaches are model-agnostic and require no fine-tuning. The question is whether they actually save tokens, how much they cost in terms of summary quality, and whether the savings depend on the domain.

We compare four conditions: baseline, input-only compression, output-only compression, and both combined, on 150 Amazon product review bundles summarized by Claude Sonnet. We measure cost (token counts at API pricing) and quality (BERTScore F1 between the summary and original text). We find that input compression reduces cost by 27% with no measurable quality loss, output compression saves 7% but drops quality slightly, and combining both yields 33% savings.

2 Method

2.1 Dataset

We use the Amazon Reviews 2023 corpus Hou et al. (2024), specifically the All Beauty category. We select 150 products that each have 10-20 reviews with a mean review length above 100 char-

acters. For each product, all reviews are concatenated into a single input document. This gives us inputs of 2,000-6,000 tokens, which is long enough for compression to matter.

2.2 Input Compression

We use spaCy (`en_core_web_sm`) to parse the text and remove tokens based on part-of-speech and lexical membership:

- Determiners, auxiliaries, conjunctions, adpositions, particles
- A manually selected set of 35 filler adverbs (“just”, “really”, “basically”, “actually”, etc.)
- 37 light verbs (“has”, “is”, “was”, “make”, “get”, etc.)
- Stop-word pronouns and adverbs
- 36 low-information adjectives (“other”, “same”, “various”, etc.)

Surviving tokens are joined with spaces; sentences are separated by periods. The approach is deliberately simple: no syntax rewriting, no paraphrasing, just deletion.

2.3 Output Compression

We prepend a system prompt: “Respond in caveman-speak. Drop articles, filler, hedging. Fragments OK. Keep all facts.” The model receives the same information but is instructed to write shorter texts, dropping stylistic padding. The idea is that summarization output often contains unnecessary hedging (“most reviewers seem to agree that”, “overall the general consensus appears to be”) which adds tokens without adding meaning. We chose caveman-speak specifically because it targets filler rather than content words, and the model should still mention every fact.

2.4 Conditions

1. **Baseline**: raw reviews in, normal summary out.
2. **Input compressed**: stripped reviews in, normal summary out.
3. **Output compressed**: raw reviews in, caveman prompt active.
4. **Both**: stripped reviews in, caveman prompt active.

The user prompt is: “Summarize these product reviews in plain text, no markdown: [text]”. Model used is Claude Sonnet.

2.5 Evaluation

We use BERTScore F1 Zhang et al. (2020) between each summary and the original concatenated reviews. BERTScore computes cosine similarity between contextual embeddings of tokens in the candidate and reference, then greedily matches them. This captures semantic overlap regardless of surface form, which is important because caveman-speak changes grammar without necessarily losing meaning. We use the default roberta-large model for scoring.

We chose BERTScore over ROUGE because ROUGE penalizes paraphrasing and would unfairly punish the output-compressed condition for using different words. We also chose it over human evaluation because we have 600 summaries (150 products $\times$ 4 conditions) and need a metric that scales.

Cost is computed at Anthropic API pricing: \$3/M input tokens, \$15/M output tokens.

3 Results

3.1 Token Reduction

Table 1 shows the main results. Input compression reduces mean input tokens from 2826 to 1661 (41% reduction). Output compression reduces mean output tokens from 297 to 232 (22% reduction). When combined, both reductions add up.

Figure 1 visualizes the token counts and cost breakdown. The cost savings come primarily from input compression (output tokens are 5 $\times$ more expensive per token but there are far fewer of them).

Table 1: Main results across 150 product summaries.

Condition	Inp.	Out.	Cost	BERT
Baseline	2826	297	\$1.94	0.831
Output compr.	2852	232	\$1.80	0.821
Input compr.	1661	300	\$1.42	0.832
Both	1687	237	\$1.29	0.820

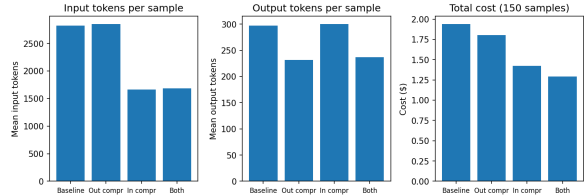


Figure 1: Mean token counts and total cost per condition.

3.2 Quality

Input compression preserves quality perfectly: BERTScore is 0.832 vs 0.831 baseline. The model reconstructs meaning from the compressed input without difficulty.

Output compression drops BERTScore from 0.831 to 0.821, a small but consistent loss. Figure 2 shows the per-sample distribution: the median shifts down and the lower tail extends further.

3.3 Cost-Quality Tradeoff

Figure 3 plots cost against quality for each condition. Input compression dominates: it moves left (cheaper) without moving down (no quality loss). Output compression and “both” move further left but also shift down. The practical question is whether 1 point of BERTScore matters for the use case.

3.4 Compression Ratio

Figure 4 shows the distribution of input compression ratios across products. The mean is 0.587 (41% token reduction), with a narrow distribution (std 0.03). This stability comes from the fact that product reviews share a similar vocabulary of filler language: “I really love this”, “honestly I have to say”, “it’s been about 3 months now”.

3.5 Qualitative Examples

To illustrate the output compression effect, here is a baseline summary of a cuticle pusher product

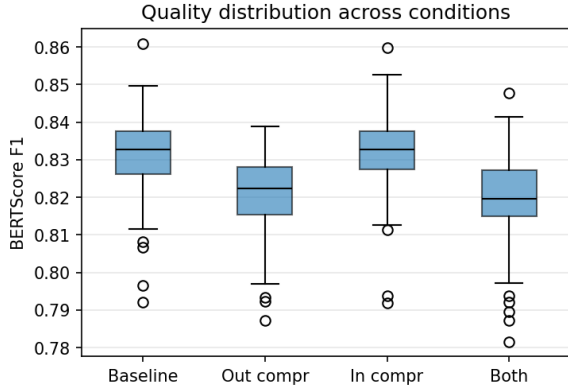


Figure 2: BERTScore F1 distribution per condition.

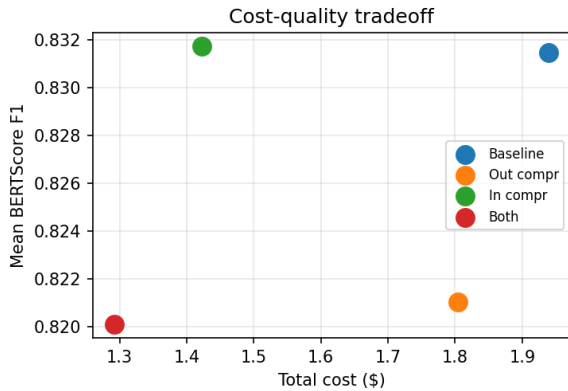


Figure 3: Cost vs. quality.

(246 tokens):

The Got Glamour cuticle pusher is a stainless steel tool that has received mostly positive reviews. Customers love that it is made in the USA, easy to sanitize, and more durable than disposable wooden or bamboo pushers. Many reviewers noted that the curved, smooth tip makes it comfortable...

The same product under output compression (138 tokens):

Stainless steel cuticle pusher. Made in USA. Most reviewers love it. Better than wood or bamboo pushers which go dull fast. Metal mean easy sanitize - dishwasher, boiling water, alcohol wipe. Curved end push cuticles back, other end clean under nails.

The caveman version is 44% shorter but retains all key facts.

4 Discussion

Input compression works for free. Reviews are full of first-person hedging, filler adverbs, and syntactic scaffolding that carry no information for

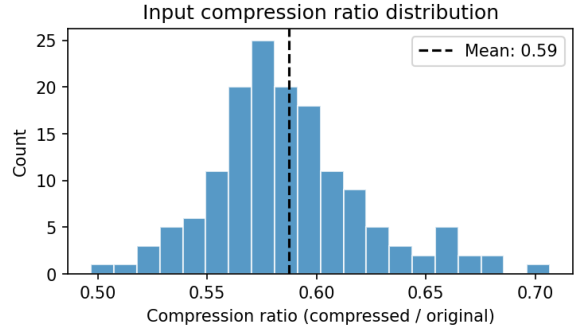


Figure 4: Distribution of input compression ratios.

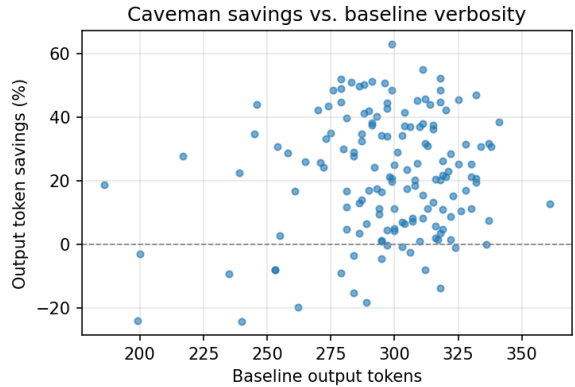


Figure 5: Per-sample output token savings from caveman prompt vs. baseline output length. Longer baseline summaries tend to benefit more.

summarization. A typical review sentence like “I honestly have to say that I really love this product” compresses to “love product”, the model only needs the content words to produce a correct summary. This works because LLMs are trained on noisy web text and handle degraded grammar without difficulty. The compressed input is still unambiguous to the model even if a little unusual to a human.

Why output compression costs quality. The BERTScore drop is not from lost facts but from reduced lexical overlap. Caveman-speak uses fewer and different words than the review text, so the embedding similarity decreases even when the meaning is preserved. For instance, “customers expressed satisfaction” and “buyers happy” carry the same information, but the second has lower token overlap with the original reviews. This is a limitation of BERTScore as a metric for style-shifted text. A human evaluator would likely rate both summaries equally.

Domain dependence. Reviews, with their opinion language and social padding (“to be hon-

est”, “I would say that overall”, “in my experience”), are a better target for output compression. We expect similar gains on customer support text or forum posts. Other domains, such as for example news articles, are already largely fact-dense, and consist primarily of names, numbers, and locations, so a caveman-style summarization would not have such a dramatic effect.

Scaling. At 150 samples the combined approach saves \$0.65 over baseline. Scaled to 100K documents, the savings would be approximately \$430. For a production pipeline processing reviews daily, input compression alone (which has no quality cost) would pay for the spaCy processing many times over. The spaCy pipeline processes roughly 10K tokens per second on a single CPU core, so the compression step adds negligible latency compared to the API round-trip.

4.1 Limitations

Our evaluation has several constraints. First, BERTScore is an imperfect proxy for summary quality: it measures semantic similarity but not factual correctness or coherence. A summary could hallucinate a fact not in the reviews and still score well if the wording is similar. Second, we test only one model (Claude Sonnet); other models may respond differently to the caveman prompt. Third, the input compression rules are English-specific and would need adaptation for other languages. Finally, we compress all part of speech categories equally aggressively: a more nuanced approach might preserve adjectives that carry sentiment (“terrible”, “amazing”) while still dropping low-information ones.

5 Conclusion

Mechanical input compression via POS-based and filler-removal filtering reduces LLM summarization cost by 27% with no quality penalty on product reviews. Prompt-based output compression adds another 7% savings at the cost of a small BERTScore drop. The two approaches are orthogonal and can be combined for a total 33% cost reduction. Input compression is the safer default for any production system; output compression is useful when the slight quality tradeoff is acceptable and the downstream consumer does not require natural-sounding text.

Reproducibility

Code, data, and outputs are available at <https://github.com/magromovoi/token-diet>. The experiment uses the Amazon Reviews 2023 dataset via HuggingFace, spaCy for compression, and Anthropic API. Full reproduction requires an API key and costs approximately \$5.

References

- Hou, Y., Li, J., He, Z., et al. (2024). Bridging Language and Items for Retrieval and Recommendation. arXiv:2403.03952.
- Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., & Artzi, Y. (2020). BERTScore: Evaluating Text Generation with BERT. In ICLR 2020.